

Mejoras de Code Smells - Iteración N 1

Daher Ezequiel - Claros, Alex - Quesada, Bautista / Proc. Desarrollo de Software 2026.

Alcance

El criterio aplicado fue mantener cambios simples y justificables: un code smell es una señal de que algo puede mejorarse, no una regla mecánica que obligue a modificar cualquier caso que se parezca superficialmente.

1. Código duplicado

Problema detectado

Se encontraron validaciones y fragmentos de lógica repetidos en distintos servicios, especialmente en:

- validación de IDs;
- validación de DNI;
- validación de nombre y apellido;
- validación de domicilio;
- validación del motivo de consulta;
- comprobación de turnos futuros;
- validaciones comunes al registrar y actualizar turnos;
- estructura repetida de `puedeAtender(...)` en las especialidades de Odontólogo;
- apertura, lectura, escritura y tratamiento de errores de los cuatro archivos de persistencia;
- escape y separación de los campos almacenados.

Cambios realizados

Se creó `ValidacionesClinica` para centralizar las validaciones reutilizadas por las distintas clases de servicio.

Se creó `HistorialTurnos`, que conserva la colección y centraliza la consulta `tieneTurnosFuturos()`. `Paciente` y `Odontologo` delegan en su historial sin exponer la lista interna.

En `TurnoValidador` se extrajo `validarDatosComunesTurno(...)`, de forma que las validaciones compartidas entre el alta y la actualización de un turno se definan una sola vez.

En `Odontologo` se agregó `motivoContieneAlgunaPalabra(...)` para reutilizar la estructura común de búsqueda de palabras dentro del motivo de consulta.

Se creó `ArchivoTexto` para centralizar la apertura de archivos, la creación de directorios, la lectura, la escritura y el tratamiento de errores.

Se creó `FormatoLinea` para que el escape, la separación y la reconstrucción de campos se definan una sola vez y sean compartidos por todos los tipos persistentes.

Resultado

Las reglas repetidas quedaron centralizadas y cualquier modificación futura puede realizarse en un único lugar, reduciendo inconsistencias y mejorando el mantenimiento.

2. Métodos largos

Problema detectado

Se encontraron métodos que concentraban demasiadas tareas dentro de una misma implementación, principalmente en `TurnoServiceImpl`, `TurnoPanel`, `PersistenciaServicio` y `Main`.

Cambios realizados

En `TurnoPanel` se dividió la construcción de la interfaz en métodos más pequeños y descriptivos, separando el armado de filas, campos, botones y filtros.

En `TurnoServiceImpl`, la modificación de un turno quedó separada en operaciones de validación y aplicación de cambios, evitando concentrar todo el flujo dentro de un único método extenso.

`PersistenciaServicio` se redujo a una fachada. La lectura, escritura y reconstrucción se distribuyeron en:

- `PersistenciaPaciente`
- `PersistenciaOdontologo`
- `PersistenciaSecretaria`
- `PersistenciaTurno`

Además, `ArchivoTexto` concentra las operaciones físicas sobre los archivos y `FormatoLinea` se ocupa del formato escapado de cada registro.

En `Main` se simplificó el armado inicial de la aplicación utilizando `DependenciasClinica`.

Resultado

Los métodos principales expresan con mayor claridad qué tarea realizan y delegan los detalles en operaciones más pequeñas, facilitando la lectura, el mantenimiento y las pruebas.

3. God Class / Long Class

Problema detectado

`TurnoServiceImpl` concentraba demasiadas responsabilidades:

- registrar turnos;
- actualizar y modificar turnos;
- cambiar estados;
- eliminar turnos;
- realizar búsquedas y filtros;
- validar datos;
- validar conflictos de horario;
- sincronizar relaciones entre entidades;
- calcular montos.

Esto hacía que una sola clase conociera y coordinara demasiadas partes del sistema.

También `PersistenciaServicio` era una clase extensa que conocía los cuatro archivos, el formato de cada entidad, la creación de objetos, la sincronización de IDs, la resolución de relaciones y las operaciones de entrada/salida.

Cambios realizados

Las responsabilidades se distribuyeron de la siguiente forma:

- `TurnoServiceImpl`: operaciones principales de agenda, como registrar, actualizar, modificar, cambiar estado y eliminar.
- `TurnoConsultaService`: búsquedas, filtros y cálculo de monto.
- `TurnoValidador`: validaciones de los datos del turno, resolución de entidades existentes y conflictos de horario.
- `Turno`: comportamiento relacionado con su propia vinculación y desvinculación con los historiales de Paciente, Odontólogo y Secretaria.

En persistencia, las responsabilidades quedaron distribuidas de esta manera:

- `PersistenciaServicio`: fachada que conserva la API pública y coordina las operaciones generales de guardado y carga;
- `PersistenciaPaciente`, `PersistenciaOdontologo`, `PersistenciaSecretaria` y `PersistenciaTurno`: conversión entre cada entidad y su representación persistida, incluida la sincronización del contador correspondiente;
- `ArchivoTexto`: acceso físico a los archivos;
- `FormatoLinea`: escape, unión y lectura de campos.

Resultado

`TurnoServiceImpl` dejó de concentrar toda la lógica del módulo y `PersistenciaServicio` pasó de más de 300 líneas a una fachada de aproximadamente 50 líneas. Cada componente tiene una responsabilidad más específica, aumentando la cohesión y reduciendo el acoplamiento.

4. Parámetros largos

Problema detectado

Se encontraron métodos que recibían una gran cantidad de datos individuales.

Los casos más claros aparecían al registrar o editar pacientes y turnos, donde los Controllers debían recibir muchos parámetros sueltos y del mismo tipo.

También `MainFrame` necesitaba recibir varias dependencias por separado.

Cambios realizados

Se agregaron objetos que agrupan los datos relacionados:

- `PacienteRegistro`
- `PacienteEdicion`
- `TurnoRegistro`
- `TurnoEdicion`

De esta forma, el Controller recibe un único objeto representativo de la operación en lugar de 6, 7, 9 o 10 parámetros independientes.

También se agregó `DependenciasClinica`, que agrupa las dependencias necesarias para construir la interfaz principal.

Como resultado, `MainFrame` recibe un único objeto `DependenciasClinica` en lugar de múltiples servicios y Controllers separados.

Casos no modificados

Los constructores de `Paciente` y `Turno` todavía tienen 6 parámetros.

Se consideran candidatos secundarios, pero no se agregó Builder ni otra abstracción solamente para reducir un parámetro. En el estado actual del proyecto, hacerlo agregaría más complejidad que beneficio. Puede revisarse nuevamente si las entidades crecen.

5. Ruptura de encapsulamiento

Problema detectado

Se encontraron cadenas de getters donde una clase necesitaba conocer la estructura interna de otra.

Ejemplos:

```
paciente.getDomicilio().getCalle()
turno.getPaciente().getNombre()
turno.getOdontologo().getId()
```

Esto generaba dependencia directa sobre cómo estaban compuestas internamente las entidades.

Cambios realizados

Se agregaron operaciones de mayor nivel para que las clases clientes pidan directamente la información que necesitan.

Ejemplos:

```
paciente.getCalleDomicilio()
turno.getNombrePaciente()
turno.getIdOdontologo()
persona.getNombreCompleto()
```

También la lógica de vincular y desvincular un turno con los historiales de sus actores quedó encapsulada dentro de **Turno**.

Se eliminaron los getters públicos que devolvían las listas de turnos de **Paciente**, **Odontologo** y **Secretaria**. Las colecciones quedaron dentro de **HistorialTurnos**, que ofrece la consulta de dominio **tieneTurnosFuturos()** sin permitir que servicios u otras capas modifiquen la lista.

Los métodos **agregarTurno(...)** y **removerTurno(...)** de los actores quedaron con acceso de paquete, por lo que la relación se modifica a través de **Turno.vincularConActores()**, **Turno.desvincularDeActores()** y los métodos de cambio de actor.

Resultado

Las vistas, repositorios, consultas y servicios necesitan conocer menos detalles internos de las entidades, reduciendo el acoplamiento estructural.

6. Intimidad inapropiada

Problema detectado

Persona exponía sus atributos como **protected**, y las subclases accedían directamente a **id**, **nombre**, **apellido** y **dni**.

Eso permitía que **Paciente**, **Odontologo** y **Secretaria** dependieran de la representación interna de su superclase.

Además, durante la carga, **PersistenciaServicio** conocía que debía agregar manualmente el mismo turno a los historiales de sus tres actores.

Cambios realizados

Se cambiaron los atributos de **Persona** a **private**.

Se reemplazaron los accesos directos en las subclases por getters como `getId()`, `getNombre()`, `getApellido()` y `getDni()`.

La reconstrucción de turnos ahora solicita `turno.vincularConActores()`. La persistencia ya no conoce cómo se mantienen internamente los historiales.

Resultado

La jerarquía quedó mejor encapsulada, las subclases dejaron de depender de la implementación interna de `Persona` y la capa de persistencia dejó de manipular detalles internos de las relaciones del dominio.

7. Herencia rechazada

Resultado del análisis

No se detectó un caso claro de herencia rechazada que requiera refactor.

`Odontologo` define un contrato abstracto mediante métodos como:

- `getTarifaBase()`
- `puedeAtender()`

Las clases `OdontologoGeneral`, `Endodoncista` y `Ortodoncista` implementan esos comportamientos mediante `@Override`, porque la tarifa y la capacidad para atender un motivo sí varían entre especialidades.

`getEspecialidad()` dejó de ser abstracto. Ahora `Odontologo` almacena un `EspecialidadOdontologica` recibido por su constructor y ofrece una única implementación concreta del getter. De esta forma, las subclases no repiten un método cuyo único propósito era devolver un valor fijo.

Esto no se considera herencia rechazada porque las subclases no están descartando o anulando un comportamiento concreto definido por el padre. Simplemente implementan un comportamiento que `Odontologo` dejó abstracto.

Además, conceptualmente las clases mantienen una relación válida de tipo "es un `Odontólogo`".

8. Lazy Class

Resultado del análisis

No se detectó en el proyecto final una clase que deba eliminarse por hacer poco o nada.

Las clases auxiliares creadas durante la refactorización tienen responsabilidades concretas:

- `TurnoValidador` centraliza reglas de validación.
- `TurnoConsultaService` concentra búsquedas y filtros.
- `ValidacionesClinica` evita repetición de reglas comunes.

- **HistorialTurnos** posee la colección de turnos y las operaciones que corresponden a esa colección.
- **ArchivoTexto** encapsula la entrada/salida común.
- **FormatoLinea** concentra un único formato compartido por cuatro archivos.
- las cuatro clases **Persistencia...** conocen el formato y la reconstrucción de un tipo de entidad distinto.

TurnoHistorialUtil sí fue eliminado luego de revisar su responsabilidad: una utilidad estática no era la dueña natural de la colección y su comportamiento quedó mejor ubicado en **HistorialTurnos**.

Por lo tanto, aunque algunas clases sean pequeñas, su existencia está justificada por una responsabilidad específica y reutilizable.

9. Complejidad artificial

Resultado del análisis

No se incorporaron patrones de diseño ni capas adicionales únicamente "por las dudas".

Las soluciones se mantuvieron simples:

- métodos auxiliares para separar responsabilidades;
- DTOs para agrupar parámetros relacionados;
- clases específicas cuando existía una responsabilidad claramente diferenciada;
- comportamiento llevado al dominio cuando correspondía al propio objeto.

La separación de persistencia responde a responsabilidades que ya existían y variaban por entidad.

PersistenciaServicio continúa siendo una fachada, por lo que el resto de la aplicación no necesita conocer todas las clases internas agregadas.

Resultado

La refactorización busca reducir complejidad y no reemplazar un problema simple por una arquitectura innecesariamente más difícil de entender.

10. Nombre de variable muy corto

Problema detectado

Se encontraron nombres de variables poco expresivos, especialmente en vistas, persistencia y recorridos de colecciones.

Ejemplos:

```
p  
o  
s
```

```
t  
d  
sb  
c  
x  
n
```

Cambios realizados

Se reemplazaron por nombres que expresan qué representa cada variable.

Ejemplos:

```
p -> paciente  
o -> odontologo  
s -> secretaria  
t -> turno  
d -> domicilio  
sb -> resultado / linea / campoActual
```

También se reemplazaron variables como **c**, **x** y **n** cuando el contexto requería un nombre más descriptivo.

Se mantuvieron nombres convencionales y locales como **i** para índices de bucles cortos y **e** para eventos o excepciones, porque su significado es inmediato dentro de ese contexto.

Resultado

El código puede entenderse con menor necesidad de seguir mentalmente qué representa cada variable.

11. Código muerto

Problema detectado

Se encontraron métodos y setters conservados sin uso real en el flujo actual del proyecto.

Cambios realizados

Se eliminó `Turno.generarMensajeRecordatorio()`, que no tenía llamadas en `src/`.

Se eliminaron `PacienteController.buscarPacientePorId()` y `PacienteController.listarPacientes()`, porque la interfaz usa los accesos ordenados y la búsqueda de datos para edición.

Se quitaron setters de historial que no tenían uso:

- `Paciente.setHistorialPaciente(...)`
- `Odontologo.setHistorialOdontologo(...)`

- `Secretaria.setHistorialSecretaria(...)`

Se quitaron setters de vínculo directo en `Turno`:

- `setOdontologo(...)`
- `setSecretaria(...)`

Después de reemplazar la representación de cobertura se eliminaron `CoberturaParticular` y `CoberturaObraSocial`, ya que el enum polimórfico `CoberturaPaciente` contiene esos dos comportamientos.

También se eliminó `TurnoHistorialUtil`: la consulta dejó de usar esta utilidad estática y pasó a la clase `HistorialTurnos`, que posee la información necesaria.

Resultado

El código quedó más cohesivo y con menos superficie muerta o peligrosa para mantener inconsistencias.

12. Data Class / Only Accessors

Problema detectado

`Domicilio` estaba modelado como una clase de datos pura: campos, getters, setters y `toString()`.

Cambios realizados

Se convirtió `Domicilio` en un objeto de valor inmutable, sin setters y con sus atributos finales.

Resultado

`Domicilio` dejó de verse como un contenedor de datos mutable y pasó a representar un valor estable del dominio.

13. Primitive Obsession

Problema detectado

La cobertura del paciente estaba representada mediante un `Boolean obraSocial`. Después de una primera mejora convivían el `Boolean` y un objeto `CoberturaPaciente`, por lo que el mismo concepto permanecía duplicado. El DTO, la vista y la persistencia todavía interpretaban `true`, `false` e índices del combo.

El estado del turno también se evaluaba con comparaciones directas contra constantes del enum.

Además, la especialidad odontológica estaba representada mediante textos como `"Odontologia General"`, `"Ortodoncia"` y `"Endodoncia"`. La vista interpretaba índices del combo para decidir qué subclase crear y tanto `OdontologoPanel` como `PersistenciaServicio` contenían su propia lógica de selección.

Cambios realizados

`CoberturaPaciente` se convirtió en un enum polimórfico con las modalidades `PARTICULAR` y `OBRA_SOCIAL`. Cada modalidad implementa `calcularMonto(...)`: la primera utiliza la tarifa base del odontólogo y la segunda devuelve el copago fijo.

Se eliminaron `CoberturaObraSocial` y `CoberturaParticular` porque su comportamiento quedó contenido en el enum. `Paciente` conserva una única representación de la cobertura y `PacienteRegistro`, `PacienteController` y `PacienteServiceImpl` trabajan con el nuevo tipo.

`PacientePanel` utiliza `JComboBox<CoberturaPaciente>` y dejó de convertir índices a valores booleanos. `Facturador` continúa delegando el cálculo en la cobertura del paciente.

También `EstadoTurno` incorporó `estaActivo()`, y `Turno.estaActivoDesde(...)` usa ese comportamiento en lugar de comparar valores sueltos. `HistorialTurnos` delega en cada turno para resolver la consulta.

Para la especialidad odontológica se realizaron los siguientes cambios:

- se creó `EspecialidadOdontologica` con los valores `GENERAL`, `ORTODONCIA` y `ENDODONCIA`;
- se creó `OdontologoFactory` para centralizar la construcción de las subclases;
- `Odontologo` ahora almacena la especialidad y su método `getEspecialidad()` devuelve el nuevo tipo de dominio;
- las subclases indican su especialidad en el constructor y dejaron de repetir `getEspecialidad()`;
- `OdontologoPanel` utiliza `JComboBox<EspecialidadOdontologica>` y dejó de interpretar índices y textos;
- `Turno` y `TurnoPanel` trabajan con la especialidad tipada;
- `PersistenciaOdontologo` dejó de construir subclases mediante un `switch` sobre textos y delega en `OdontologoFactory`.

Compatibilidad con la persistencia anterior

Los nuevos archivos guardan identificadores estables: `GENERAL`, `ORTODONCIA` y `ENDODONCIA`.

La carga continúa aceptando los textos anteriores `Odontologia General`, `Ortodoncia` y `Endodoncia`. La conversión se encuentra centralizada en el método `convertirEspecialidadPersistida(...)` de `PersistenciaOdontologo` y también acepta los identificadores nuevos.

Para la cobertura, los nuevos archivos guardan `PARTICULAR` u `OBRA_SOCIAL`. La carga continúa aceptando `false` como `PARTICULAR` y `true` como `OBRA_SOCIAL`; esta conversión se encuentra centralizada en `PersistenciaPaciente.convertirCoberturaPersistida(...)`.

Resultado

La especialidad dejó de depender de textos e índices mágicos en la vista y de decisiones duplicadas de construcción. La creación de odontólogos quedó centralizada y las capas clientes trabajan con un tipo explícito del dominio.

La cobertura dejó de estar duplicada y ya no depende de un `Boolean` en la entidad, el DTO, el controlador, el servicio, la vista o el guardado nuevo. `EstadoTurno.estaActivo()` continúa encapsulando la comprobación de estados activos.

Pruebas realizadas

- compilación completa de los archivos de `src` con `javac --release 17 -encoding UTF-8`;
- verificación no interactiva de la conversión de los tres textos históricos y los tres identificadores nuevos;
- verificación de que `OdontologoFactory` crea una instancia cuya especialidad coincide con cada valor del enum;
- verificación de la conversión de `true`, `false`, `PARTICULAR` y `OBRA_SOCIAL`;
- verificación del monto particular y del copago de obra social mediante `Facturador`;
- prueba integral de guardado y carga de pacientes, las tres especialidades, secretaria y turno;
- comprobación de que la carga reconstruye el historial mediante `Turno.vincularConActores()`;
- comprobación de historiales con turnos activos, cancelados, pasados y desvinculados;
- comprobación de escape y recuperación de separadores, barras y saltos de línea en los datos persistidos;
- comprobación de sincronización de los contadores de IDs después de cargar;
- prueba integral de compatibilidad cargando los valores históricos de cobertura y especialidad;
- prueba sobre la interfaz Swing real registrando y visualizando las tres especialidades y ambas coberturas;
- edición desde la interfaz de paciente, odontólogo y turno, incluido el bloqueo de la especialidad durante la edición;
- cálculo desde la interfaz de los montos particular y de obra social;
- comprobación del mensaje de validación y del bloqueo al intentar eliminar un paciente con un turno futuro;
- eliminación confirmada desde la interfaz de un paciente sin turnos asociados;
- guardado desde el diálogo de cierre, reapertura de la aplicación y verificación visual de los datos recargados;
- búsqueda de `switch`, índices mágicos y creación directa de subclases relacionadas con especialidad fuera de la fábrica;
- búsqueda de `Boolean obraSocial`, `getObraSocial()`, `setObraSocial()` y combos de cobertura basados en texto.

La prueba funcional de la interfaz Swing correspondiente a estos cambios fue completada sin errores.